What your team actually changes

You are improving a learned next-state predictor. The host website is the visual test bench; your Python notebook trains your own weights. Start from the weak neural starter, not the host solution.

First 10 minutes at the laptop

1. Unzip the complete student kit, keeping assets/ beside student/.
2. Open team-notebook.ipynb in student/. Run the diagnostic cells.
3. Read only the model definition and loss in train_repair.py, and features() in neural_model.py. The notebook handles loading and scoring.
4. Write a hypothesis before editing. Keep the seed, scene set, action pool and score fixed.

From a terminal in student/:

```
python run_experiment.py --name baseline
python run_experiment.py --name contacts --contact-weight 5 --compare baseline
```

The first command trains a plain neural baseline. The second changes only collision-loss weight. Both use seed 42, 40 epochs, the same trajectory split and the same evaluation. The tool prints the scores, changed settings, new successes and new failures. In rehearsal this weighting change made things worse; your job is to investigate why, not assume every hint is a fix.

For an architecture experiment after diagnosing free-motion drift, compare:

```
python run_experiment.py --name free-motion --free-head --compare baseline
```

Inspect the code behind this option: it fits an affine neural layer from free-motion observations and routes those states to that layer at inference. It does not copy friction constants. Explain why this could reduce recursive error, then evaluate collisions and all targets. This is a guided implementation example; you can edit or replace the learned branch.

Each run is saved under results/runs//. Names cannot overwrite existing runs. network.json holds actual learned weights, network.report.json records training, and score.json records every evaluated scene. Choose a new run name for each change.

Choose one investigation

Observation or hypothesis	Concrete edit	What could disprove the hypothesis?
Rare collisions contribute too little to training loss	In train_repair.py, adjust `weights` or change how batches sample contacts. The contact-weight option tests the first idea.	Contact error improves but free motion or final target success worsens.
Raw x/y coordinates make geometric relationships hard to learn	In neural_model.py, inspect features(); enable `--geometry`, then propose a more useful representation.	Better one-step loss fails to improve recursive planning.

Observation or hypothesis	Concrete edit	What could disprove the hypothesis?
Training never covers the speeds being searched	Keep the same model and use `--data expanded`. Report the extra data budget.	Wide-speed planning still fails despite improved validation on those speeds.
The network cannot represent the dynamics accurately enough	Edit the nn.Sequential layers or try `--width 64`, holding other settings fixed.	A wider network does not reduce held-out error or improve the task.
Smooth-motion bias accumulates over many steps	Advanced: `--free-head` fits an affine neural layer using only free-motion training observations. Inspect its least-squares fit and routing in the code, rather than copying simulator constants.	Free rollouts improve but optimistic collision predictions still dominate planning.
Similar bumper collisions appear different after rotation	Advanced: express velocity relative to the surface normal and tangent, train in that frame and transform predicted changes back.	Rotated versions of the same local event remain inconsistent.
Accurate individual steps drift when composed	Advanced: train on several consecutive steps from a training trajectory, preserving the trajectory split.	Multi-step validation improves while the fixed planning task still regresses.

The switches are convenient experiment controls. The actual coding exercise is explaining and editing the loss, representation, sampling or architecture. You do not need to explore the entire repository. For more advanced architectural changes, keep the same callable N-by-4 next-state interface and have a host review export/inference compatibility.

See your own result

In the website, choose **Try your team's trained model**, then select your run's network.json. It stays in your browser tab. The selector changes to Your team neural network; Find a shot and Execute now use your learned predictions and the unchanged simulator. The supplied starter format supports raw or geometry inputs and hidden widths up to 256. Different feature layouts or architectures need a matching adapter; Python scoring remains the authoritative path.

Compare all 24 development scenes, not just the opening shot. A new success and a new failure both belong in your report. Submit one chosen run folder, your edited code and a short hypothesis/evidence report. Hosts run one final evaluation after submission freeze.

Everyone has a job

The operator runs code. The experiment designer chooses one variable. The skeptic specifies a disconfirming result. The recorder tracks configurations and failures. Other reviewers inspect error plots and predict outcomes before each run. Rotate the operator. Each of two hosts handles five teams and receives one structured report per team.